
Decision Support Databases 25-26

Notes

Contents

1	Introduction	2
1.1	Information Systems	3
1.2	Data Warehouses	3
1.2.1	Data Warehousing Architectures	4
1.2.2	What to Model	5
2	Information Modeling	7
2.1	Information Modeling	7
2.1.1	What to Model	8
2.2	Modeling for Operational Databases	9
2.3	Modeling for Data Warehouses	12
2.3.1	Dimensional Fact Model	12

Chapter 1

Introduction

In modern society, organizations accumulate large amounts of data, stored and managed by computers via a DBMS (Database Management System). However, this data is often underutilized, and important decisions are still based on intuition rather than informed decisions derived from data analysis. Decision support information systems are specific types of information systems designed for summarizing large amounts of data in a form that is easy to interpret. Analysis performed by such systems goes beyond simple queries executed by traditional DBMSs on operational databases, which are optimized for on-line transaction processing. Organizations maintain a separate database, called data warehouse, specifically organized for decision support applications.

Data, Information, Knowledge

Data is a representation of facts, concepts, or instructions without context, which can be processed by computers.

Information is data (raw or in a condensed form) interpreted within a certain context.

Knowledge is information that expands the recipient's capability of understanding reality, allowing them to make predictions, decisions, and take actions.

Each organization has a set of specific goals to pursue, and to achieve these goals, it uses a **structure**, a set of **resources**, and a set of **processes** that transform resources into goods and services. Processes can be classified in three categories according to the **Anthony triangle**: **strategic activities** (for long-range planning), **tactical activities** (for short term planning and control), and **operational activities** (for day to day operations). For any process, information is a key resource; different organizations will have different information needs, depending on their structure, resources, and processes. Information systems are designed to satisfy these needs.

1.1 Information Systems

Information System

An **information system** is an organized collection of resources, people, and procedures finalized to collect, store, process, and communicate the information needed to support the on-going activities of an organization.

A computerized information system is a type of information system where information is stored and processed by a computer. From now on, we will simply refer to computerized information systems as information systems.

Information systems can be classified into three categories:

- **Operational:** data is organized in a database, which is managed by a traditional DBMS and interacted with using transactions. This type of system is used for day-to-day operations;
- **Decision Support:** data is organized in a dedicated data warehouse, managed by a specialized DBMS. This type of system is used for Business Intelligence applications, i.e., to search for useful insight to support decision making. DSS have been developed in the late 70's to help managers in making decisions of three types:
 - **Structured:** for which a well-defined decision-making procedure exists;
 - **Unstructured:** for which a well-defined decision-making procedure does not exist, so that the decision will only depend on manager experience;
 - **Semi-structured:** when the decision-making procedure is partly defined, so it also requires manager input.
- **Web-based:** for E-commerce, to bring the business to the Web.

1.2 Data Warehouses

Data Warehouse

A **data warehouse** is a subject-oriented, integrated, nonvolatile, and time-varying collection of data in support of management's decisions.

A data warehouse is a specialized type of database that stores data by subject, not by applications (*subject-oriented*), includes data from multiple heterogeneous sources and merged in a coherent form (*integrated*), is not changed interactively via transactions, but is updated periodically to add or delete data (*nonvolatile*), and it contains historical data that describes the state of the enterprise at different points in time, unlike operational databases where data is kept up-to-date (*time-varying*). A special type of data warehouse is the **data mart**, which is focused on data regarding only one subject and is usually smaller in size.

Three types of decision support can be provided. **Reports**, which are simple presentations of data in a structured format, **multidimensional analysis**, which also allows users to interactively explore data from different perspectives, and **data mining/exploratory data analysis**, where algorithms are used to automatically discover interesting patterns or relationships in data.

The reasons for separating operational databases from data warehouses are mainly two: first, complex data analysis queries would degrade the performance of operational DBMSs, which are optimized for on-line transaction processing; second, the two systems have different structures, contents, and uses, since operational databases do not store any historical data and do not integrate different sources. Traditional applications that use databases are called **OLTP (On-Line Transaction Processing)**, while decision support applications that use data warehouses are called **OLAP (On-Line Analytical Processing)**.

1.2.1 Data Warehousing Architectures

The process of bringing data from operational database sources into a single data warehouse is called **data warehousing**. In practice, three types of solutions are adopted.

Single-Layer Architecture There is only one layer of data managed by the operational system. The data warehouse is defined as a view on the operational database, but is not a physically separate component. This solution is very low-cost and easy to implement, and as such is often used by small organizations. Of course, it does not actually separate operational and analytical operations, so it is not suitable for large organizations.

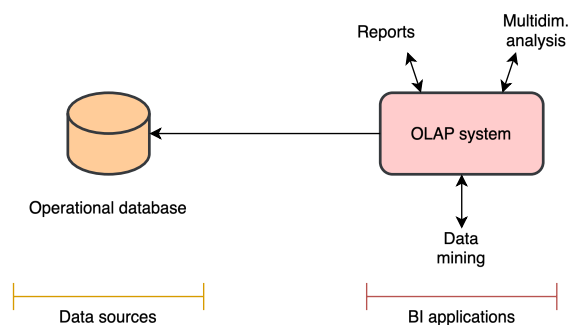


Figure 1.1: Single-Layer Architecture.

Two-Layer Architecture The data warehouse is a separate component from the operational database and is managed by its dedicated DBMS. The warehouse is loaded with data via **ETL (Extract, Transform, Load)** applications from the operational database and any other external source, which also brings this data in a consistent format and updates it periodically.

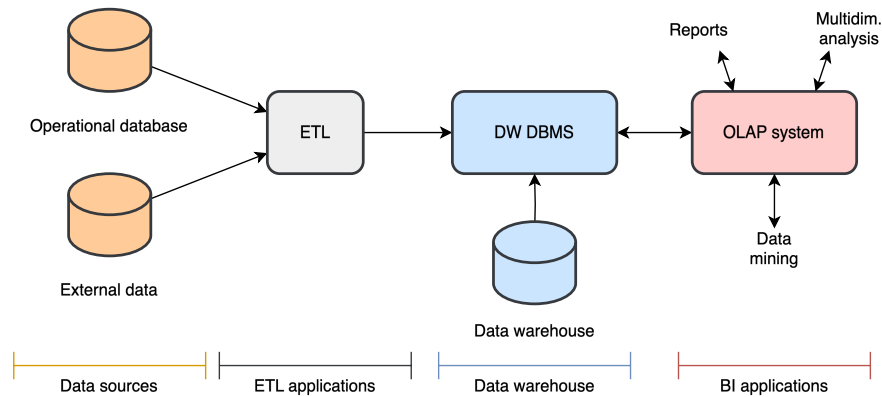


Figure 1.2: Two-Layer Architecture.

Three-Layer Architecture The third layer is called **data staging**. It contains data obtained via integration in the ETL process and prepares it to be loaded into the data warehouse, and may be implemented at different levels of complexity (it can be as simple as a set of files and as complex as a fully developed relational database). In this solution, extraction and integration is separated from reorganization and loading.

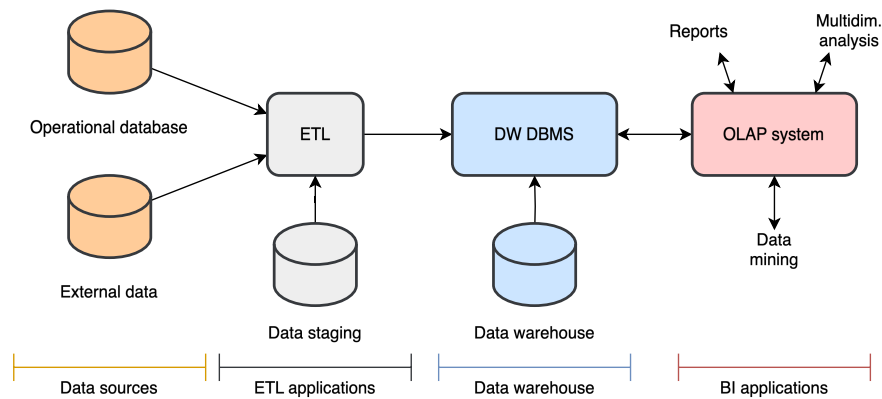


Figure 1.3: Three-Layer Architecture.

1.2.2 What to Model

Data must be organized taking into account how managers will use it to support their decisions about the performance of key business processes. More precisely, the relevant informa-

tion to model are:

- **Facts:** managers are interested in analyzing facts (i.e., observations) about the performance of a process;
- **Measures:** facts are accompanied by numerical attributes called measures (such as quantities, sizes, costs, revenues, etc.);
- **Dimensions:** managers think in terms of business dimensions, which give facts a context to make them meaningful. A dimension is often described by a set of attributes. If this is not true, it is called a *degenerate dimension*;
- **Metrics:** measures are analyzed via summaries called metrics, obtained by applying aggregation functions over facts by certain dimensions or combinations of dimensions. Common metrics are about economic performance, but when they relate to efficiency and quality they are called **Key Performance Indicators (KPI for short)**.

Aggregation functions are classified in three categories:

- **Distributive:** $sum()$, $count()$, $max()$, $min()$.
 - **Algebraic:** $avg()$, $stddev()$, $var()$.
 - **Holistic:** $median()$, $mode()$, $rank()$.
- **Hierarchies:** managers are also interested in analyzing metrics at different levels of detail, by exploiting the fact that certain dimensions have a set of attributes that form a hierarchy (e.g., date can be broken down into year, semester, quarter, month, day).

A hierarchy always implies a functional dependency between the attributes involved.

Functional Dependency

Given a relation schema R and X, Y subsets of attributes of R , a functional dependency $X \rightarrow Y$ is a constraint that specifies that for every possible instance r of R and for any two tuples $t_1, t_2 \in r$, $t_1[X] = t_2[X]$ implies $t_1[Y] = t_2[Y]$.

In simpler terms, a functional dependency $X \rightarrow Y$ means that any time we know the value that a tuple takes in X we can uniquely determine the value it takes in Y . In the date example, there is the following functional dependency: $\text{day} \rightarrow \text{month} \rightarrow \text{quarter} \rightarrow \text{semester} \rightarrow \text{year}$. Note that in order for the hierarchy to hold, the day attribute must also include the specific month and year, the month attribute must include the year, and so on.

Chapter 2

Information Modeling

This chapter discusses modeling methods and techniques for both operational databases and data warehouses, introducing the main notation elements of the most common models.

2.1 Information Modeling

When a database system is created, it is based on some model that reproduces the essential characteristics of the real world information it is intended to represent. In general, models can be either *iconic*, if they retain physical characteristics of the entities they represent, or *symbolic*, if they use symbols to represent entities and their relationships. In the context of information systems, symbolic models are used.

Symbolic Model

A **symbolic model** is a subjective, formal representation of ideas and knowledge about some aspects of the real world, called domain of discourse, designed to serve a specific purpose.

A model simplifies reality by focusing on what is important for the task at hand, and uses a formal language to express this simplified representation.

Modeling in databases is broken down into three levels of abstraction:

- **Conceptual level**, in which the basic structure of the database is defined at an high-level, after analyzing the problem and considering any user requirement. Some examples of conceptual models are the **Entity-Relationship (ER)** model and the **Object Data Model (ODM)**;
- **Logical level**, where the conceptual model is mapped to a more detailed description of the data structures, including all attributes and constraints. The model is still

independent from any specific DBMS. An example of logical model is the **Relational Data Model**;

- **Physical level**, where the physical data structures are defined for the elements in the logical model. The choices done at this level must take in consideration the features of the DBMS used to implement the database.

2.1.1 What to Model

The reality to be modeled is considered in terms of concrete knowledge, abstract knowledge, procedural knowledge, dynamics, and communications.

Concrete Knowledge Concrete knowledge refers to specific facts known of the reality to be represented. We can assume reality consists of *entities*, *properties*, and *relationships*.

Entity, Property, Relationship

An **entity** is anything for which certain facts should be recorded, independently of the existence of other entities.

A **property** is a fact about an entity which is not meaningful in itself, but only because it describes an entity of interest.

A **relationship** is a fact which correlates independent entities.

For example, we want to model the administration system of a library. Examples of entities can be a book, a bibliographic description, a registered user, or a loan record. Properties may be a user's name and address, or a book's title and author. Entities with the same properties are said to have the same **type**, and are classified in the same **collection**, or **entity set**. The books titled "1984", "Moby Dick", and "The Great Gatsby" are all part of the same entity set, and are all of type *book*. As for relationships, examples could be the relationship between a bibliographic description and one or more books (the same book may be owned multiple times by the library), or between a user and their loan records.

Abstract Knowledge Abstract knowledge concerns facts which impose certain restrictions on admissible values of concrete knowledge and on the way these values can evolve over time, or expresses rules on how to derive new information. These rules are called **integrity constraints**. In the library example, abstract knowledge could include rules about the data type of attributes, such that book properties *title* and *author* must be strings, while *length* must be a non-negative integer; another example may be the rule that a loan can only be borrowed for two weeks at a time, and must be associated with the *id* of a registered user; the age of a user can be automatically calculated by subtracting the *birthdate* from the current

date. Relationships can also have constraints that limit the possible correlated entities. The most important ones are **structural constraints**, which are:

- **Cardinality**, *one* or *many*, to specify how many entities of one collection can be associated with entities of another collection, e.g., a bibliographic description can be associated with *many* book copies, while a book copy is associated with only *one* bibliographic description, so the relationship is called *one-to-many*;
- **Participation**, *total* or *partial*, to specify whether an entity of one collection can have entities of another collection associated with it, e.g., a loan record must be associated with one registered user, so the participation from the loan record side is *total*; on the other hand, a registered user may have never loaned a book, so the participation from the registered user side is *partial*.

2.2 Modeling for Operational Databases

To construct the conceptual model for an operational database, we define a **schema**, a collection of time-invariant definitions which model the structure of admissible data (including integrity constraints), and procedural knowledge (i.e., the elementary operations applied to concrete knowledge to cause changes). Different formalisms, each with a specific data model, can be used to define the schema.

Data Model

A **Data Model** is a set of abstraction mechanisms, with associated operators and implicit integrity constraints, used to define a database schema.

A data model is a symbolic model specific for databases. A type of data model is the **Object Data Model**. The basic mechanisms of the ODM are:

- **Objects**: they are the equivalent of entities. An object is a software entity with an internal state represented by instance variables, and a behaviour described by a set of methods. Each object has its own identity that persists over time, even if its internal state change;
- **Object types**: just like an entity belongs to a type, an object belongs to an object type. An object type describes the state fields and the implementation of methods common to all objects of that type;
- **Classes**: a modifiable sequence of objects with the same type. They are the equivalent of entity sets. Each class introduces the definition of a type and a constructor for values of this type, as well as a name to denote the class itself;

- **Relationships:** classes have relationships with other classes, which are characterized by a cardinality and partiality. A relationship may also have attributes, or be recursive (i.e., a class has a relationship with itself);
- **Inheritance:** a mechanism that allows something to be defined by only describing how it differs from a previously defined one. It is distinct from subtyping, which is a relation between types such that when one type A is a subtype of another type B , then any operation applicable to type B can also be applied to type A (although inheritance is one way to define subtypes). A type can be defined from a single supertype (*single inheritance*) or from several supertypes (*multiple inheritance*).
- **Class hierarchies:** similarly to types, classes can also be organized in hierarchies. If C_1 is a subset of C_2 , then C_1 is a *subclass* of C_2 , and this implies that:
 - The type of elements in C_1 is a subtype of the type of the elements in C_2 ;
 - The elements in C_1 are a subset of the elements in C_2 , and C_1 inherits any relationship defined on C_2 .

When the same superclass is shared by two or more subclasses, two constraints can be defined: **overlap** and **covering**. A no overlap constraint specifies that two subclasses cannot share any element. In the absence of this constraint, the same object may belong to more than one subclass. A covering constraint specifies that the objects in the subclasses must collectively include all elements in the superclass. In the absence of this constraint, the superclass may have elements that do not belong to any subclass. When both constraints are present, we call the hierarchy a **generalization**.

The following figures illustrate a possible graphical notation used for ODM. There is no standard notation; the one used here is based on the notation used in UML.

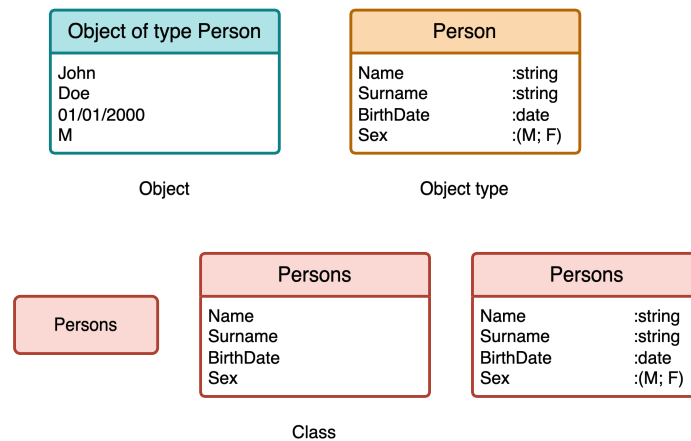


Figure 2.1: Graphical notation for objects, object types, and classes. Classes can be shown with different levels of detail.

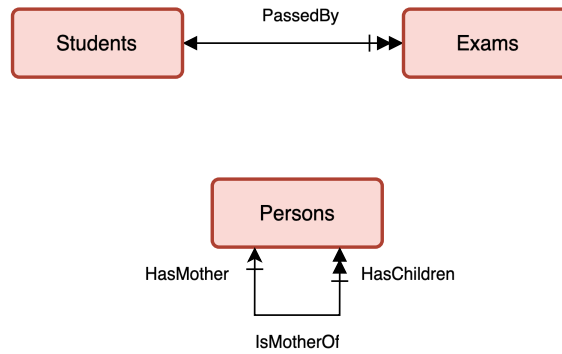


Figure 2.2: Simple relationships between classes. Each relationship is represented by an arc with a label. Multivalued relationships are represented with a double arrow. Optional relationships have a crossed line. Recursive relationships require labels on the ends of the arcs to clarify the meaning.

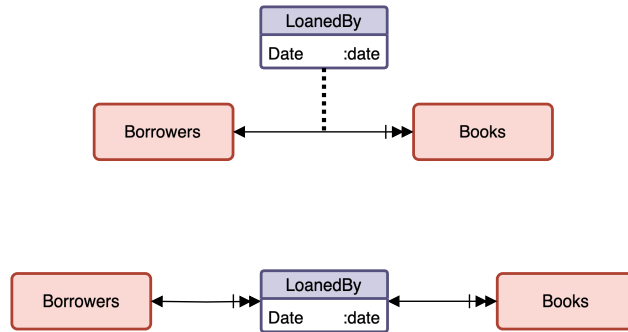


Figure 2.3: Relationships with attributes.

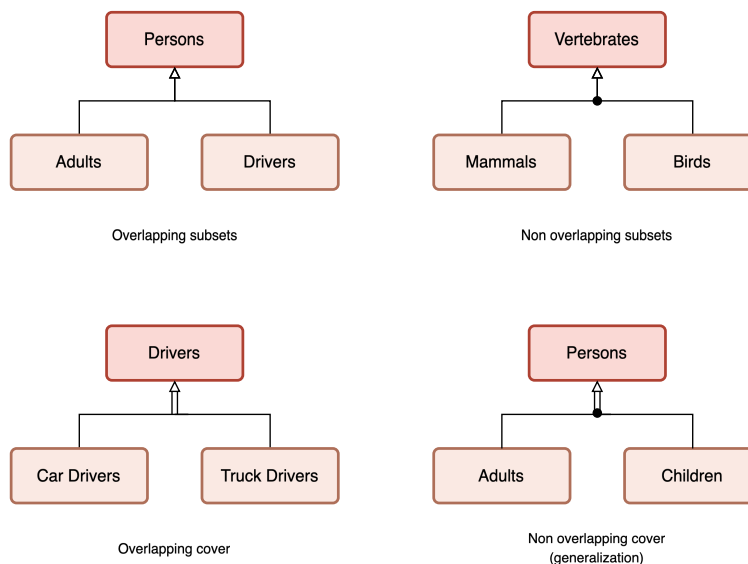


Figure 2.4: Class hierarchies.

2.3 Modeling for Data Warehouses

This section will introduce three models to define the structure of a data warehouse: for the conceptual level, the **Dimensional Fact Model (DFM)**, while for the logical level, the **Relational Data Model** and the **Multidimensional Cube Model**.

2.3.1 Dimensional Fact Model

A data warehouse is based on a multidimensional model, i.e., a model that takes in consideration multiple business dimensions to allow the user to explore aggregated information across many possible dimensional combinations to get a full perspective on the data. The Dimensional Fact Model is a graphical conceptual model for data warehouses. The following are its basic elements.

Facts The most important abstraction mechanism of the conceptual model is the collection of facts, which are observations of the performance of a business model. Facts are modeled as rectangles divided in two parts: the top contains the fact name, while the bottom contains a set of **measures**. A measure is a numerical property of a fact that describes one of its quantitative aspects of interest for analysis. Facts that are not associated with any measure are called *factless fact*, although to keep terminology consistent, we will call them **measureless facts**. This type of fact is defined when there is only interest in counting its number of instances.

Facts can be of different nature:

- **Transaction facts** represent information on a specific event that occurred at a specific point in time during the execution of a business process, e.g., a withdrawal on a bank account.
- **Periodic snapshot facts** represent the information on a series of events that have occurred over a period of time, e.g., the monthly summary of all transactions on a bank account.
- **Accumulating snapshot facts** represent the information on the lifetime of an evolving event that has a duration and a change over time, e.g., a mortgage application which is done in several steps (presenting the documents, undergoing bank review, approval/refusal, completion).

Also measures can be classified in different types:

- **Additive/flow/rate measures** can be meaningfully aggregated with the function *sum* by any dimension. This is the most common type of measure. An example may be the number of products sold in a day.

- **Semi-additive/stock/level measures** can be meaningfully aggregated with the function *sum* by certain dimensions, but not all. These measures refer to a particular point in time and are used to record the state of the event: for example, the monthly inventory quantity of a certain product. We say that a measure is semi-additive with respect to a dimension D_1 if it cannot be aggregated with the function *sum* for groups of data with different values of D_1 .
- **Non-additive/value-per-unit measures** cannot be aggregated with the function *sum* by any dimension. These measures are usually the result of ratios, so the only meaningful operation is counting the number of facts with a specific value. Other common non-additive measures are per-unit prices, measures of intensity (e.g., a temperature), or averages.
- **Calculated measures** are derivated from other measures. These are used to avoid having users perform these calculations, such as calculating revenue from product price minus discount.



Figure 2.5: Graphical representation of facts.

Dimensions Dimensions give facts a context. Graphically, they are represented by lines starting from the corresponding fact and ending with a circle. A dimension may also be described by a set of attributes, which are drawn as lines ending with a circle connected to the “main” circle of the dimension. Each attribute is also labeled with its unique name, and the same name should not be used for attributes of different dimensions.

In some cases, an interesting aspect to model is a hierarchical relationship between attributes. Using relational data model terminology, a hierarchy can be defined when there is a functional dependency between attributes: for example, if we define the dimension *Date*, with attributes *Day*, *Month*, *Year*, then we can define the hierarchy $Day \rightarrow Month \rightarrow Year$. The same attribute may even appear in multiple hierarchical structures for the same dimension: if we also add the attribute *Week*, then we can define $Week \rightarrow Year$ (this is because a week can overlap multiple months, so it does not belong to the previous dependency chain). A dimension without attributes is called **degenerate**.

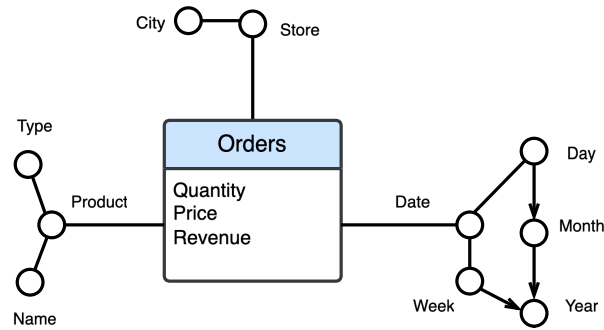


Figure 2.6: Graphical representation of dimensions.

The key steps to design a DFM schema are the following:

1. Gather requirements, identifying the key elements to insert in the model.
2. Identify the granularity of the fact, i.e., the level of detail at which the fact will be represented. Granularity determines which measures and dimensions will appear in the model. As a general rule, it is best to choose a fine grain, even if it increases the number of facts to be treated, since later on, aggregation functions can always reduce the level of detail.
3. Identify the fact measures. Not everything that is numeric is necessarily a measure.
4. Identify the dimensions and dimensional attributes. To discern dimensions in the original requirements analysis, it is useful to consider the classic 5W-1H questions.
5. If possible, identify any dimensional attribute hierarchy.